

Lesson 1: NumPy Fundamentals

Cơ bản về NumPy

Lesson Information | Thông tin bài học

Item	Description
Duration	90 minutes
Type	Lecture + Lab
Prerequisites	Module 1: Python Basics

Learning Objectives | Mục tiêu bài học

#	Objective	Assessment
1	Understand NumPy arrays and their advantages	Quiz
2	Create arrays using various methods	Code exercises
3	Perform basic array operations	Lab work
4	Use indexing and slicing effectively	Code exercises

1. Introduction to NumPy (15 min)

1.1 What is NumPy?

NumPy (Numerical Python) is the fundamental package for scientific computing in Python.

Key Features:

- N-dimensional array object (ndarray)
- Broadcasting functions
- Linear algebra, random numbers, Fourier transform
- Integration with C/C++ and Fortran code

1.2 Why NumPy?

Feature	Python List	NumPy Array
Speed	Slow	10-100x faster
Memory	High	Efficient
Operations	Element-by-element	Vectorized
Functionality	Basic	Advanced math

```
# Speed comparison
import numpy as np
import time

# Python list
python_list = list(range(1000000))
start = time.time()
result = [x * 2 for x in python_list]
print(f"Python: {time.time() - start:.4f}s")

# NumPy array
numpy_array = np.arange(1000000)
start = time.time()
result = numpy_array * 2
print(f"NumPy: {time.time() - start:.4f}s")
# NumPy is typically 10-100x faster!
```

1.3 Installation and Import

```
pip install numpy
```

```
import numpy as np # Standard convention
print(np.__version__)
```

2. Creating Arrays (25 min)

2.1 From Python Lists

```
# 1D array
arr1d = np.array([1, 2, 3, 4, 5])
print(arr1d)          # [1 2 3 4 5]
print(type(arr1d))    # <class 'numpy.ndarray'>

# 2D array (matrix)
arr2d = np.array([[1, 2, 3],
                  [4, 5, 6]])
print(arr2d)
# [[1 2 3]
#  [4 5 6]]

# 3D array
arr3d = np.array([[[1, 2], [3, 4]],
                  [[5, 6], [7, 8]]])
print(arr3d.shape)    # (2, 2, 2)
```

2.2 Built-in Array Creation Functions

```
# zeros - all zeros
zeros = np.zeros((3, 4))
print(zeros)
# [[0. 0. 0. 0.]
#  [0. 0. 0. 0.]
#  [0. 0. 0. 0.]]

# ones - all ones
ones = np.ones((2, 3))
print(ones)
# [[1. 1. 1.]
#  [1. 1. 1.]]

# full - all same value
full = np.full((2, 3), 7)
print(full)
# [[7 7 7]
#  [7 7 7]]

# empty - uninitialized (faster but random values)
empty = np.empty((2, 2))

# identity matrix
identity = np.eye(4)
print(identity)
# [[1. 0. 0. 0.]
#  [0. 1. 0. 0.]
#  [0. 0. 1. 0.]
#  [0. 0. 0. 1.]]

# diagonal matrix
diag = np.diag([1, 2, 3, 4])
```

2.3 Sequence Generation

```
# arange - like range but returns array
arr = np.arange(10)          # [0 1 2 3 4 5 6 7 8 9]
arr = np.arange(2, 10)       # [2 3 4 5 6 7 8 9]
arr = np.arange(0, 10, 2)     # [0 2 4 6 8]
arr = np.arange(10, 0, -1)    # [10 9 8 7 6 5 4 3 2 1]

# linspace - evenly spaced numbers
arr = np.linspace(0, 1, 5)    # [0.  0.25 0.5  0.75 1.  ]
arr = np.linspace(0, 10, 11) # [0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]

# logspace - evenly spaced on log scale
arr = np.logspace(0, 3, 4)     # [1. 10. 100. 1000.]
```

2.4 Random Arrays

```
# Set seed for reproducibility
np.random.seed(42)

# Random floats [0, 1)
rand = np.random.rand(3, 4)

# Random floats from standard normal distribution
randn = np.random.randn(3, 4)

# Random integers
randint = np.random.randint(0, 10, size=(3, 4))

# Random choice from array
choice = np.random.choice([1, 2, 3, 4, 5], size=10)

# Shuffle array
arr = np.arange(10)
np.random.shuffle(arr)

# Random sample from distributions
normal = np.random.normal(loc=0, scale=1, size=1000)
uniform = np.random.uniform(low=0, high=1, size=1000)
```

3. Array Attributes (10 min)

```
arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

# Shape - dimensions
print(arr.shape)      # (3, 4) - 3 rows, 4 columns

# Number of dimensions
print(arr.ndim)       # 2

# Total elements
print(arr.size)       # 12

# Data type
print(arr.dtype)      # int64

# Item size in bytes
print(arr.itemsize)   # 8
```

```
# Total bytes
print(arr.nbytes)      # 96 (12 elements × 8 bytes)
```

Data Types

```
# Specify dtype
arr_int = np.array([1, 2, 3], dtype=np.int32)
arr_float = np.array([1, 2, 3], dtype=np.float64)
arr_complex = np.array([1, 2, 3], dtype=np.complex128)
arr_bool = np.array([1, 0, 1], dtype=np.bool_)

# Common dtypes
# int8, int16, int32, int64
# uint8, uint16, uint32, uint64
# float16, float32, float64
# complex64, complex128
# bool_

# Convert dtype
arr = np.array([1.5, 2.7, 3.9])
arr_int = arr.astype(np.int32) # [1 2 3]
```

4. Indexing and Slicing (25 min)

4.1 Basic Indexing

```
# 1D array
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])      # 10 (first element)
print(arr[-1])     # 50 (last element)
print(arr[2])      # 30

# 2D array
arr2d = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

print(arr2d[0, 0]) # 1 (row 0, col 0)
print(arr2d[1, 2]) # 6 (row 1, col 2)
print(arr2d[2, -1]) # 9 (row 2, last col)

# Alternative syntax
print(arr2d[0][0]) # 1 (works but slower)
```

4.2 Slicing

```

arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

# Basic slicing [start:stop:step]
print(arr[2:7])      # [2 3 4 5 6]
print(arr[:5])       # [0 1 2 3 4]
print(arr[5:])       # [5 6 7 8 9]
print(arr[::2])      # [0 2 4 6 8]
print(arr[::-1])     # [9 8 7 6 5 4 3 2 1 0] (reverse)

# 2D slicing
arr2d = np.array([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9, 10, 11, 12]])

print(arr2d[0, :])   # [1 2 3 4] - first row
print(arr2d[:, 0])   # [1 5 9] - first column
print(arr2d[0:2, 1:3]) # [[2 3] [6 7]] - submatrix
print(arr2d[::2, ::2]) # [[1 3] [9 11]] - every other

```

4.3 Boolean Indexing

```

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Create boolean mask
mask = arr > 5
print(mask) # [False False False False False True True True True True]

# Apply mask
print(arr[mask]) # [6 7 8 9 10]

# Shorthand
print(arr[arr > 5]) # [6 7 8 9 10]
print(arr[arr % 2 == 0]) # [2 4 6 8 10] - even numbers

# Multiple conditions
print(arr[(arr > 3) & (arr < 8)]) # [4 5 6 7]
print(arr[(arr < 3) | (arr > 8)]) # [1 2 9 10]

# Modify with boolean indexing
arr[arr > 5] = 0
print(arr) # [1 2 3 4 5 0 0 0 0 0]

```

4.4 Fancy Indexing

```

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90])

# Index with array of indices
indices = np.array([1, 3, 5, 7])

```

```

print(arr[indices]) # [20 40 60 80]

# 2D fancy indexing
arr2d = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

# Select specific elements
rows = np.array([0, 1, 2])
cols = np.array([2, 1, 0])
print(arr2d[rows, cols]) # [3 5 7] (diagonal from top-right)

# Select specific rows
print(arr2d[[0, 2]]) # [[1 2 3] [7 8 9]]

```

5. Array Manipulation (15 min)

5.1 Reshaping

```

arr = np.arange(12)
print(arr) # [ 0  1  2  3  4  5  6  7  8  9 10 11]

# Reshape
reshaped = arr.reshape(3, 4)
print(reshaped)
# [[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]]

# Use -1 for automatic dimension
reshaped = arr.reshape(3, -1) # (3, 4)
reshaped = arr.reshape(-1, 6) # (2, 6)

# Flatten - convert to 1D
flattened = reshaped.flatten() # Returns copy
raveled = reshaped.ravel()     # Returns view (faster)

# Transpose
transposed = reshaped.T
print(transposed.shape) # (4, 3)

```

5.2 Stacking and Splitting

```

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

# Vertical stack
vstack = np.vstack((a, b))

```

```

print(vstack)
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]

# Horizontal stack
hstack = np.hstack((a, b))
print(hstack)
# [[1 2 5 6]
#  [3 4 7 8]]

# Concatenate (general)
concat = np.concatenate((a, b), axis=0) # Same as vstack
concat = np.concatenate((a, b), axis=1) # Same as hstack

# Split
arr = np.arange(12).reshape(4, 3)
split = np.vsplit(arr, 2) # Split into 2 along rows
split = np.hsplit(arr, 3) # Split into 3 along columns
split = np.split(arr, 2, axis=0) # General split

```

5.3 Adding and Removing Elements

```

arr = np.array([1, 2, 3])

# Append
new_arr = np.append(arr, [4, 5, 6]) # [1 2 3 4 5 6]

# Insert
new_arr = np.insert(arr, 1, [10, 20]) # [1 10 20 2 3]

# Delete
new_arr = np.delete(arr, 1) # [1 3]

# Unique
arr = np.array([1, 2, 2, 3, 3, 3])
unique = np.unique(arr) # [1 2 3]
unique, counts = np.unique(arr, return_counts=True)

```

Practice Exercises | Bài tập thực hành

Exercise 1.1: Array Creation

```

# Create the following arrays:
# 1. Array of 20 zeros
# 2. 5x5 identity matrix
# 3. Array from 10 to 50 (step 5)

```



```
# 4. 4x4 array of random integers (0-100)
# 5. 3x3 array with values from 1 to 9
```

Exercise 1.2: Indexing Practice

```
# Given:
arr = np.arange(1, 26).reshape(5, 5)

# Extract:
# 1. Element at row 2, column 3
# 2. Last row
# 3. First column
# 4. Submatrix (rows 1-3, columns 2-4)
# 5. Diagonal elements
# 6. Elements greater than 15
```

Exercise 1.3: Checkerboard

```
# Create an 8x8 checkerboard pattern:
# [[0 1 0 1 0 1 0 1]
#  [1 0 1 0 1 0 1 0]
#  [0 1 0 1 0 1 0 1]
#  ...]
```

Exercise 1.4: Array Manipulation

```
# 1. Create array 1-12, reshape to 3x4
# 2. Transpose it
# 3. Flatten it
# 4. Split into 3 equal parts
# 5. Stack them horizontally
```

Summary | Tóm tắt

Key Functions

Function	Description
<code>np.array()</code>	Create from list
<code>np.zeros()</code>	All zeros
<code>np.ones()</code>	All ones
<code>np.arange()</code>	Range of values

Function	Description
<code>np.linspace()</code>	Evenly spaced
<code>np.random.rand()</code>	Random [0,1)
<code>np.reshape()</code>	Change shape
<code>np.vstack/hstack()</code>	Stack arrays

Key Attributes

Attribute	Description
<code>.shape</code>	Dimensions
<code>.ndim</code>	Number of dimensions
<code>.size</code>	Total elements
<code>.dtype</code>	Data type

Next Lesson | Bài tiếp theo

Lesson 2: NumPy Advanced Operations

- Array arithmetic
- Broadcasting
- Aggregation functions
- Linear algebra